UDEC
UNIVERSIDAD DE CUNDINAMARCA

Semana 4:

Pruebas de Seguridad en API

Las APIs juegan un rol crucial en las aplicaciones modernas, ya que permiten la comunicación entre diferentes sistemas y servicios, facilitando el intercambio de datos de manera eficiente. Sin embargo, su amplia adopción también las convierte en un objetivo atractivo para los atacantes. Por este motivo, es fundamental garantizar que las APIs estén adecuadamente protegidas.

En este documento profundizaremos en la **autenticación**, **autorización**, **protección de datos** y **prevención de ataques comunes** en APIs.

OWASP API Security Project

El **OWASP API Security Project** es un recurso clave para los desarrolladores y profesionales de seguridad interesados en proteger las APIs. Este proyecto ofrece un conjunto de mejores prácticas y guías para asegurar APIs y prevenir ataques comunes.

Principales Riesgos Identificados en OWASP API Security Project:

- **Autorización rota a nivel de objetos (BOLA):** Los atacantes podrían intentar explotar endpoints de la API que son vulnerables a la autorización rota a nivel de objetos. Podrían manipular los identificadores de

objetos dentro de una solicitud para obtener acceso no autorizado a datos sensibles.

- **Autenticación rota**: Si la autenticación se implementa incorrectamente, los atacantes podrían hacerse pasar por usuarios de la API y acceder a datos confidenciales.
- **Autorización rota a nivel de propiedades de objetos**: La falta de autorización o una validación incorrecta de la autorización a nivel de las propiedades de los objetos podría dejar información expuesta o vulnerable a la manipulación por parte de personas no autorizadas.
- **Consumo de recursos sin restricciones**: Muchas APIs no limitan las interacciones del cliente o el consumo de recursos. Los atacantes podrían generar un alto volumen de solicitudes a la API, lo que podría aumentar los costos operativos y llevar a una denegación de servicio.
- **Autorización rota a nivel de funciones**: Los atacantes podrían enviar solicitudes API legítimas a un endpoint al que no deberían poder acceder. Esto les permitiría obtener acceso a los recursos o funciones administrativas de otros usuarios.
- **Acceso sin restricciones a flujos de negocio sensibles**: Las APIs podrían exponer un flujo de negocio (como publicar un comentario en un sitio web, comprar un producto o hacer una reserva), lo que permitiría a los atacantes ejecutar ese flujo de manera excesiva.

- **Server-side request forgery (SSRF)**: Una API podría obtener un recurso remoto sin validar la URL proporcionada por el usuario. Por ejemplo, un usuario podría proporcionar una URL para cargar una imagen en una plataforma de redes sociales. Esa URL podría iniciar un escaneo de puertos dentro de una red interna.
- **Gestión inadecuada del inventario**: Las APIs pueden exponer más endpoints que las aplicaciones web tradicionales. Si las organizaciones no llevan un inventario de los hosts y las versiones de API implementadas, podrían dejar versiones de API obsoletas y endpoints vulnerables.
- **Consumo inseguro de APIs**: Los atacantes podrían atacar servicios de terceros que interactúan con APIs en lugar de atacar una API directamente. Saben que los desarrolladores a menudo confían más en los datos de las APIs de terceros que en los datos proporcionados por los usuarios.

Recomendaciones para Proteger APIs

- **Autenticación Segura**: Utilizar protocolos de autenticación robustos como **OAuth2** y **JWT (JSON Web Tokens)**. Estos protocolos permiten asegurar que solo usuarios autenticados accedan a los recursos protegidos de la API.

- **Autorización**: Aplicar un control de acceso basado en roles (RBAC) o basado en atributos (ABAC) para garantizar que cada usuario solo acceda a los recursos que tiene permiso para ver o modificar.

- **Cifrado de datos**: Todas las comunicaciones entre el cliente y la API deben estar cifradas utilizando **TLS (Transport Layer Security)**, garantizando que los datos no sean interceptados o alterados por atacantes durante la transmisión.

- **Rate limiting y monitoreo**: Implementar **limites de tasa** (rate limiting) para prevenir ataques de fuerza bruta y abuso de la API. Además, se debe monitorear continuamente el tráfico y el uso de la API para detectar comportamientos anómalos.

Pruebas de Seguridad en APIs

Las **pruebas de seguridad en APIs** son fundamentales para identificar vulnerabilidades que pueden comprometer la integridad, confidencialidad y disponibilidad de los sistemas. A continuación, se describen los pasos clave para realizar pruebas de seguridad en APIs, abarcando temas como **autenticación**, **autorización**, **protección de datos sensibles**, y la prevención de **ataques comunes**.

1. Pruebas de Autenticación

El primer paso es verificar la implementación correcta de los mecanismos de autenticación en la API. Una autenticación débil o rota podría permitir a los atacantes acceder a recursos protegidos.

Ejemplos de pruebas:

- **Validación de credenciales**: Enviar solicitudes con credenciales incorrectas o incompletas para comprobar si la API permite o bloquea el acceso.
- **Fuerza bruta de contraseñas**: Realizar intentos repetidos de autenticación con diferentes combinaciones de contraseñas para verificar si hay medidas de protección como bloqueos temporales o uso de captchas.
- **Manipulación de tokens JWT**: Intentar modificar el contenido de un token JWT para acceder a recursos no autorizados. Verificar si la API valida correctamente la firma y los permisos asociados al token.
- **Autenticación multifactor (MFA)**: Si la API utiliza MFA, comprobar si es posible eludir este mecanismo o si está correctamente implementado.

2. Pruebas de Autorización

Las pruebas de autorización son esenciales para garantizar que solo los usuarios autorizados puedan acceder a los recursos y realizar acciones específicas dentro de la API.

Ejemplos de pruebas:

- **Pruebas de escalación de privilegios**: Simular un intento de acceder a recursos o realizar acciones que deberían estar restringidas para usuarios con menos privilegios. Por ejemplo, un usuario con rol de cliente intenta acceder a funcionalidades administrativas.

- **Acceso a datos de otros usuarios (Broken Object Level Authorization - BOLA)**: Intentar acceder a datos o recursos de otros usuarios mediante la manipulación de parámetros de la URL. Por ejemplo, si una API permite acceder a detalles de pedidos con la URL /api/orders/{order_id}, probar si cambiando el order_id se pueden acceder a los pedidos de otros usuarios.

- **Pruebas de función**: Intentar realizar llamadas API a funciones que no deberían estar disponibles para el usuario autenticado, como funciones administrativas.

3. Protección de Datos Sensibles

Las APIs suelen manejar datos sensibles como información personal, credenciales de acceso o datos financieros. Estas pruebas aseguran que la API proteja adecuadamente estos datos tanto en tránsito como en reposo.

Ejemplos de pruebas:

- **Revisión de respuestas de la API**: Verificar si la API devuelve más información de la necesaria en las respuestas, como datos sensibles que no deberían ser visibles (contraseñas, números de seguridad social, etc.).

- **Pruebas de cifrado de datos en tránsito**: Usar herramientas como **Wireshark** para analizar si las comunicaciones entre el cliente y la API están cifradas adecuadamente utilizando **HTTPS (TLS)**.

- **Pruebas de filtración de datos a través de errores**: Probar si los mensajes de error de la API devuelven información sensible que podría ser utilizada para planificar ataques.

4. Prevención de Ataques Comunes

Las APIs son vulnerables a una serie de ataques comunes. Estas pruebas ayudan a identificar y mitigar las vulnerabilidades antes de que puedan ser explotadas por atacantes.

Ejemplos de pruebas:

- **Inyección SQL**: Probar si la API es vulnerable a inyección de SQL enviando consultas maliciosas en los parámetros de entrada. Herramientas como **SQLmap** pueden automatizar estas pruebas.

- **Cross-Site Scripting (XSS)**: Enviar entradas maliciosas en los parámetros o formularios de la API para ver si las respuestas reflejan esas

entradas sin filtrarlas, lo que permitiría ejecutar scripts maliciosos en el navegador del usuario.

- **Denegación de Servicio (DoS)**: Realizar ataques de denegación de servicio enviando un gran número de solicitudes simultáneas para verificar si la API implementa medidas de **rate limiting** o control de solicitudes.

- **Server-Side Request Forgery (SSRF)**: Probar si la API puede ser engañada para realizar solicitudes hacia recursos internos o externos mediante la manipulación de URLs proporcionadas por el usuario.

5. Integración de Pruebas en el Ciclo de Desarrollo

Para maximizar la efectividad de las pruebas de seguridad, estas deben integrarse en el ciclo de desarrollo continuo de la API. Las pruebas deben automatizarse y ejecutarse con cada cambio en el código para garantizar que no se introduzcan nuevas vulnerabilidades a lo largo del tiempo.

Ejemplos de integración:

- **Automatización de pruebas**: Utilizar herramientas de **integración continua (CI)**, como **Jenkins** o **GitLab CI**, para ejecutar las pruebas de seguridad de forma automática cada vez que se implementen nuevas versiones de la API.

- **Monitoreo continuo**: Implementar herramientas de monitoreo, como **ELK Stack** o **Splunk**, para analizar y registrar el tráfico API y detectar comportamientos sospechosos.

6. Herramientas de Pruebas de Seguridad en APIs

- **OWASP ZAP**: Una herramienta gratuita y de código abierto para probar la seguridad de las aplicaciones web y APIs. Permite interceptar y modificar solicitudes, realizar análisis de vulnerabilidades y simular ataques.

- **Burp Suite**: Otra herramienta poderosa utilizada para realizar análisis de seguridad en APIs. Permite interceptar, modificar y automatizar pruebas para descubrir vulnerabilidades como inyección de SQL o XSS.

- **Postman**: Aunque principalmente es una herramienta de desarrollo para probar APIs, también puede ser utilizada para realizar pruebas básicas de seguridad, como autenticación, manejo de tokens y pruebas de autorización.

7. Informe Final de Pruebas

Al finalizar las pruebas, es importante documentar los resultados de manera exhaustiva para que el equipo de desarrollo pueda implementar las correcciones necesarias.

El informe debe incluir:

1. **Descripción de la API**: Información sobre los endpoints probados y su funcionalidad.

2. **Vulnerabilidades identificadas**: Detalle de las vulnerabilidades encontradas, su gravedad y el impacto potencial.

3. **Pruebas realizadas**: Explicación de las pruebas ejecutadas, incluyendo ejemplos de solicitudes y respuestas.

4. **Recomendaciones**: Sugerencias para corregir las vulnerabilidades identificadas y mejorar la seguridad general de la API.

5. **Herramientas utilizadas**: Lista de las herramientas empleadas para realizar las pruebas.

Ejemplo de practica de seguridad en APIs

Práctica: Pruebas de Seguridad en API

En este ejemplo se realizan pruebas de seguridad en una API pública de ejemplo, utilizando herramientas y técnicas para identificar posibles vulnerabilidades relacionadas con la autenticación, autorización, manejo de datos sensibles y exposición a ataques comunes como inyección de código y ataques de denegación de servicio. La API utilizada será **JSONPlaceholder**, una API pública diseñada para pruebas y simulación de operaciones CRUD.

Objetivo:

El objetivo de esta práctica es aplicar pruebas de seguridad a una API pública, identificando posibles vulnerabilidades y generando un informe con los hallazgos y recomendaciones de mejora.

Herramientas:

- **Postman**: Para enviar solicitudes API y analizar respuestas.
- **OWASP ZAP** o **Burp Suite**: Para interceptar y analizar el tráfico entre el cliente y la API.
- **Wireshark**: Para inspeccionar el tráfico de red y verificar si los datos están correctamente cifrados.
- **SQLmap**: Para probar la API contra inyecciones SQL (si aplica).
- **Insomnia** (opcional): Otra herramienta para realizar pruebas manuales de API.

Procedimiento:

Paso 1: Configuración del Entorno de Pruebas

1. **Descargar e instalar Postman o Insomnia**.
   - Postman e Insomnia permiten enviar y recibir solicitudes HTTP para probar la API. Estos pueden manejar diferentes métodos de HTTP como GET, POST, PUT, y DELETE.

2. **Configurar OWASP ZAP o Burp Suite**.

   - Estas herramientas interceptarán las solicitudes y respuestas entre el cliente (Postman/Insomnia) y la API para examinar el tráfico y permitir modificaciones en tiempo real.

3. **Instalar Wireshark**.

   - Wireshark será utilizado para analizar si las comunicaciones entre la API y el cliente están cifradas mediante TLS.

Paso 2: Realizar Pruebas de Autenticación

Objetivo: Verificar que los mecanismos de autenticación implementados en la API son seguros y no pueden ser manipulados.

1. **Solicitar acceso a un recurso protegido**:

   - Usando Postman, intenta acceder a un recurso que requiera autenticación (si la API lo soporta).

   - Verifica si puedes acceder sin autenticación o si la API devuelve un error de acceso no autorizado (401 Unauthorized).

2. **Prueba de fuerza bruta en el login**:

- Si la API tiene un endpoint de inicio de sesión, intenta hacer una prueba de fuerza bruta enviando varias combinaciones de nombres de usuario y contraseñas.
- Usar OWASP ZAP o Burp Suite para automatizar el envío de múltiples solicitudes. Verificar si la API implementa protección contra estos ataques, como bloquear la cuenta después de varios intentos fallidos o aplicar captchas.

3. **Manipulación de tokens**:

- Si la API utiliza tokens JWT para la autenticación, captura el token mediante OWASP ZAP o Burp Suite. Intenta modificar el payload o la firma del token para acceder a recursos restringidos.
- Verifica si la API valida correctamente los tokens y detecta modificaciones no autorizadas.

Paso 3: Pruebas de Autorización

Objetivo: Garantizar que los controles de autorización se aplican correctamente y que los usuarios no puedan acceder a recursos de otros usuarios o realizar acciones fuera de su rol.

1. **Pruebas de acceso a objetos de otros usuarios**:

- En un endpoint que devuelva datos específicos de un usuario (por ejemplo, /api/orders/{order_id}), intenta modificar el order_id para acceder a los datos de otro usuario.
- Verifica si la API permite acceder a datos que no deberían estar disponibles para el usuario autenticado.

2. **Pruebas de escalación de privilegios**:
   - Simula un intento de acceder a un recurso o función administrativa desde una cuenta con privilegios limitados. Por ejemplo, intenta acceder a un endpoint de administración con un rol de usuario estándar.
   - Verifica si la API responde con un código de error (403 Forbidden) o permite el acceso.

Paso 4: Pruebas de Exposición de Datos Sensibles

Objetivo: Comprobar que la API no expone más datos de los necesarios en sus respuestas y que los datos sensibles están correctamente protegidos.

1. **Revisión de respuestas API**:
   - En Postman, realiza solicitudes a diferentes endpoints de la API. Analiza las respuestas y verifica si incluyen información confidencial no necesaria (como contraseñas, tokens o datos financieros).

2. **Verificación de cifrado en tránsito**:

   - Usar Wireshark para capturar el tráfico entre el cliente y la API.
   - Asegúrate de que todas las comunicaciones están cifradas mediante HTTPS (TLS) y que no se transmiten datos en texto claro.

3. **Exposición de datos a través de mensajes de error**:

   - Genera errores en las solicitudes (por ejemplo, enviando parámetros incorrectos o malformados) para ver si la API devuelve mensajes de error que contienen información confidencial, como nombres de tabla o detalles del sistema.

Paso 5: Pruebas de Prevención de Ataques Comunes

Objetivo: Detectar vulnerabilidades comunes que podrían ser explotadas por atacantes, como inyección de código o denegación de servicio.

1. **Inyección SQL**:

   - Si la API utiliza bases de datos, prueba los endpoints que aceptan parámetros de consulta, como /api/search?q=keyword.
   - Usa **SQLmap** para automatizar pruebas de inyección SQL e intenta ejecutar consultas maliciosas que podrían permitir acceso no autorizado a la base de datos.

2. **Cross-Site Scripting (XSS)**:

   - Envía entradas maliciosas a la API que se reflejen en las respuestas. Verifica si la API filtra correctamente los datos de entrada para evitar que los scripts maliciosos se ejecuten en los navegadores de los usuarios.

3. **Denegación de Servicio (DoS)**:

   - Envía múltiples solicitudes simultáneas a la API utilizando **OWASP ZAP** o **Burp Suite** para verificar si la API tiene mecanismos de **rate limiting** o límites de uso de recursos para protegerse de ataques de denegación de servicio.

Paso 6: Documentación de Resultados

1. **Redactar un informe de vulnerabilidades**:

   - Al finalizar las pruebas, documenta todas las vulnerabilidades identificadas, incluyendo:

     - Descripción del problema.
     - Gravedad de la vulnerabilidad (baja, media, alta).
     - Recomendaciones para corregir el problema.

2. **Incluir ejemplos de solicitudes y respuestas**:

- Proporciona ejemplos concretos de las solicitudes que realizaste y las respuestas obtenidas de la API para demostrar las vulnerabilidades encontradas.

3. **Conclusiones**:

- Resume el estado de seguridad de la API y ofrece recomendaciones generales sobre cómo mejorar su seguridad.

Referencias:

1. Checkmarx. (n.d.). *CXSAST datasheet*. Checkmarx. Recuperado de https://checkmarx.com/resources/data-sheet/cxsast-datasheet/
2. Ekran System. (n.d.). *Top 10 cyber security breaches*. Ekran System. Recuperado de https://www.ekransystem.com/en/blog/top-10-cyber-security-breaches
3. Eviden. (2024). *Top 10 cybersecurity threats*. Eviden. Recuperado de https://eviden.com/publications/digital-security-magazine/cybersecurity-predictions-2024/top-10-cybersecurity-threats/
4. Karisma Foundation. (2022, noviembre). *Ciberataque a Sanitas: Impactos diferenciales sobre mujeres cuidadoras*. Fundación Karisma.

Recuperado de https://web.karisma.org.co/ciberataque-a-sanitas-impactos-diferenciales-sobre-mujeres-cuidadoras/#:~:text=En%20noviembre%20de%202022%2C%20las,o%20de%20difundir%20la%20informaci%C3%B3n

5. SonarSource. (n.d.). *SonarQube documentation*. SonarSource. Recuperado de https://docs.sonarsource.com/sonarqube/latest/?_gl=1*1bqrg5r*_gcl_au*MTg0ODkwNDc1LjE3MjY4NDk3NzU.*_ga*MzcyNjM0NDUyLjE3MjY4NDk3NzU.*_ga_9JZ0GZ5TC6*MTcyNjg0OTc3NS4xLjEuMTcyNjg0OTk0OS41Mi4wLjA.